

1. Data Retrieval

1.1 API Selection & Endpoint Configuration

We used the most desirable Open Electricity REST [API](https://api.openelectricity.org.au) endpoints on the NEM network to retrieve structured data, avoiding less efficient alternatives. The selected endpoints (each with <https://api.openelectricity.org.au> in the front) are:

[/v4/facilities/](#): Getting the facility code, facility name and their corresponding latitude and longitude for later processing in the frontend Visualization.

[/v4/data/facilities/NEM](#): For per-facility 5-minute interval power generation and CO₂ emissions data.

[/v4/market/network/NEM](#): For per-market 5-minute interval price and demand data.

All endpoints were filtered to target the NEM network (Queensland, NSW, Victoria, South Australia, Tasmania) via the [network=NEM](#) parameter.

1.2 Data Time Range & Rate Limiting Compliance

Time Interval & Duration: We retrieved data for the last 7 consecutive days in October 2025 (2025-10-25 to 2025-10-31) w, fitting the rubric's minimum requirement of "1 week" and ensuring sufficient data volume for streaming. All records were forced to a 5-minute interval via the [interval=5min](#) parameter, with no gaps in time granularity.

While the Open Electricity API lacks explicit strict rate limit documentation, we implemented defensive measures to avoid request throttling: we used reusable [requests.Session\(\)](#) objects to reduce connection overhead, and capped concurrent requests at 15 workers in multi-threaded batch facility data fetching. It also fits the Optimization and Efficiency requirement since it takes about 2-4 min to retrieve all the data in that interval. This balance of efficiency and stability ensured smooth data retrieval without triggering API access restrictions.

2. Data Preprocessing & Integration with A1

2.1 Data Describe

We integrated multiple heterogeneous datasets to form a unified foundation aligned with Assignment 1's facility metadata, including: facility base data (from Open Electricity API's [/v4/facilities/](#), with [facility_code](#), [facility_name](#), [lat/lng](#)), facility-specific operational data (from [/v4/data/facilities/NEM](#), covering 5-minute Power (MW) and Emissions (tonnes)), market-wide data (from [/v4/market/network/NEM](#), with 5-minute Price (\$/MWh) and Demand (MW)), and Assignment 1's NGER/CER datasets (providing [primaryFuel](#), [futureFuelSource](#), and [state](#)). The final consolidated dataset uses [facility_code](#) as the primary integration key, with [timestamp](#) standardized from **Unix** or timestamp **without timezone** to **ISO 8601** (Sydney timezone, UTC+11) for consistency. The final consolidated dataset unifies these sources with the following Schema:

Column Name	Data Type	Source Endpoint	Purpose
facility_code	String	/v4/facilities/	Unique identifier for facilities, used as the primary key for integration and data retrieve
facility_name	String	/v4/facilities/	The name for each facility for latter mapping with Assignment1 data
lat/lng	Numeric	/v4/facilities/	Longitude and latitude, Geographic coordinates for spatial visualization
timestamp	ISO 8601	All endpoints	A ISO 8601 time interval within 5 min interval in the current Sydney time zone aka. UTC +11.
Power (MW)	Numeric	/v4/data/facilities/NEM	Instantaneous power output/consumption (MW)
Emission (tonnes)	Numeric	/v4/data/facilities/NEM	CO2 equivalent emissions (tonnes)
Price (\$/MWH)	Numeric	/v4/market/network/NEM	Price per unit of energy (\$/MWh)
Demand (MW)	Numeric	/v4/market/network/NEM	Demand for energy (MW)
state	String	NGER/CER datasets (Assignment 1)	Administrative region of the facility
fuel_list	String	NGER datasets (Assignment 1)	Current and planned fuel technology for using

2.2 Comprehensive Data Cleaning

Data cleaning was treated as a foundational step to eliminate noise and ensure reliability for downstream streaming and analysis, with targeted solutions for each data quality issue.

For anomalies, we addressed invalid negative values in core metrics first. Since power output and emissions are inherently non-negative, we audited and counted negatives in **Power (MW)** and **Emissions (tonnes)** (excluding NaN to avoid false positives), then used vectorized `mask()` operations to replace negatives with 0—preserving NaN for genuine missing data. Post-validation confirmed no negatives remained, eliminating distorted calculation risks.

Missing value processing used a tiered strategy: Facilities with full missing **Power (MW)** and **Emissions (tonnes)** were filtered out for lacking operational insights. For continuous gaps in these 5-minute interval time-series columns, we split each segment and filled the first half via forward fill, the second via backward fill—minimizing trend distortion. Missing optional market data (**Price (\$/MWh)**, **Demand (MW)**) was retained as NaN to avoid artificial market trend smoothing.

Data type standardization fixed raw API inconsistencies: Mixed timestamps (Unix epoch, strings) were unified to ISO 8601 (e.g., `2025-10-25T00:00:00+11:00`) with Sydney timezone alignment using `pd.to_datetime(..., format='ISO8601')`. All metrics were cast to float to preserve decimal precision and avoid truncation.

Finally, deduplication via the `(facility_code, timestamp)` composite key removed duplicates with `pandas.drop_duplicates(keep="first")`, ensuring unique 5-minute records. The API's `network_id=NEM` parameter filtered out non-NEM facilities, aligning with the assignment's scope.

2.3 Integration with Assignment 1 Data

We used a fuzzy matching pipeline to merge operational data with Assignment 1's static metadata: A `combine_matching` function matched via `facility_name` (left table: API facility list, right tables: NGER/CER datasets), retaining all right-table matches for each left-table facility and preserving non-matching left-table facilities (with NaN for metadata). Redundant `lat/lng` from NGER/CER were dropped to avoid conflicts. We then grouped the dataset by `facility_name`, `facility_code`, `lat`, `lng`, `state`, and aggregated `primaryFuel` (all values) and non-null `futureFuelSource` into a unified `fuel_list` via a custom function. The final dataset was saved as `data_for_publish.csv`, validated for no missing headers/malformed rows, optimized for streaming, and compatible with downstream processing.

2.4 Preprocessing Outcomes

Post-cleaning and integration: Key metric columns had no invalid negative values; partial missing power/emissions data was filled accurately; Assignment 1's static metadata (state, fuel technologies) was linked to operational data; the dataset maintained a 5-minute interval with ISO 8601 timestamps, ready for MQTT streaming and dynamic visualization.

3. Data Publishing via MQTT

We implement a streaming publisher that emits **combined power+emissions** messages for each facility as a **single, strictly ordered reliable event stream**. The stream is ordered by **event time** and enforces an **exact 0.1-second delay** between consecutive messages.

3.1 Schema and Topics

Each dynamic record is published as a single composite message to the topic `comp5339/task123/measurements/{facility_code}`, containing all the fields from cleaned integration dataset `consolidated_data_cleaned.csv` required by downstream tasks: `facility_code`, `timestamp` (standardized to RFC-3339 format), `power_value`(MW), `emission_value` (tCO₂-e), `price_per_mwh` and `demand_mw`, a small `unit` object, and a `source` tag. To make the stream auditable and easy to verify the 0.1-second delay, the message body also includes two time markers: `slot_mono_ns`, representing the scheduled emission time used to enforce a 0.1-second frequency; and `sent_mono_ns`, the actual monotonic clock at the time of sending. For static facility metadata (name and coordinates) from `facility_list.csv`, is published once per facility to the topic `comp5339/task123/facility_info/{facility_code}`, which also keep all fields set so that subscriber can immediately access the contextual information.

3.2 Event Time Order and Delay Enforcement

Chronological order and reliability: We have established a deterministic total order of event times across all facilities. Before emission, each record is globally sorted according to `(timestamp, facility_code)` to ensure that the interleaving order across facilities strictly follows the chronological order and is reproducible. During the replay cycle, we use a lexicographical cursor `(last_ts, last_fac)`: if a record's timestamp/ facility code is greater than the last timestamp/ facility code, the record is considered a new record. This ensures that historical data will not be republished (only new data is published), and avoids the tail drop issue when multiple rows share the last processed timestamp. Each message under MQTT QoS=1 with a persistent

client session, subscribers can observe a continuous and ordered message stream even in the event of brief disconnections.

0.1-Second Delay: The interval between messages is enforced through an absolute timeslot schedule rather than relative sleep time. At the start of a cycle, we determine a monotonic start time t_0 , and the target timeslot for the k -th message is $t_0 + (k + 1) \times 0.1$ seconds. The publisher reaches this precise boundary via coarse sleep and short spin, and follows a strict no-chasing rule: if delayed, it jumps forward to the next timeslot to ensure the interval is always ≥ 0.1 seconds and never smaller. On Windows, I called `timeBeginPeriod(1)` to improve timer precision and help scheduling stability. Each payload records both the scheduled slot time (`slot_mono_ns`) and the actual send time (`sent_mono_ns`), making the interval fixed at 0.1000 seconds with sub-millisecond accuracy.

4. Data Subscribing & Integration with Assignment 1

4.1 Correct Subscription & Decoding

The system implements a high-performance MQTT subscriber using the `paho-mqtt` library, strictly aligned with Task 3's publishing configuration. It subscribes to the wildcard topic `comp5339/task123/measurements/#` to capture real-time data for all facilities, ensuring no messages are missed.

To avoid interface blocking during high-frequency (0.1s) message streams, the subscriber uses asynchronous callback mechanisms:

The `on_connect` callback verifies successful MQTT connection (return code `rc=0`), then subscribes to the target topic; connection failures are logged with error codes for troubleshooting.

The `on_message` callback decodes JSON payloads, parses all required fields (including `facility_code`, `lat`, `lng`, `power_value`, `emission_value`, and Assignment 1-derived `state/fuel_list`), and validates core data integrity—skipping records missing critical fields (e.g., no `facility_code` or invalid coordinates) to avoid downstream errors.

Valid data is dual-stored for persistence and real-time rendering:

In-memory cache: Stored in Streamlit's `session_state.facility_data` (keyed by `facility_code`) for instant dashboard updates, with numeric metrics rounded to 2 decimal places for readability.

Permanent storage: Appended to `nem_facility_data.csv` (with headers auto-added if the file is new), enabling historical data restoration on dashboard restart.

4.2 Integration with Assignment 1

Integration leverages `facility_code` as the unified primary key, linking real-time MQTT streaming data with static metadata from Assignment 1 (NGER/CER datasets), ensuring 1:1 mapping accuracy:

Static metadata retention: Assignment 1-derived fields (`state`, `fuel_list`, `lat`, `lng`, `facility_name`) are pushed into the mqtt server merge with facility `measurements` data.

Dynamic data merging: Real-time metrics (`power_value`, `emission_value`, `price_per_mwh`, `demand_mw`, `timestamp`) from MQTT are merged with the corresponding static record in `session_state` on each message receipt. This ensures static attributes (e.g., fuel type, region) stay consistent while dynamic metrics refresh in real time.

Invalid data exclusion: Facilities without valid coordinates (from Assignment 1's `lat/lng` fields) or complete metadata are filtered out during MQTT processing, ensuring only geographically actionable records are visualized on the map.

The integration maintains metadata consistency with Assignment 1's schema, while enabling real-time updates for operational metrics—fulfilling the requirement to unify historical static data with live streaming data.

5. Dynamic Visualization

5.1 Dashboard Framework & Core Functionality

The real-time electricity dashboard, built with Streamlit and Folium, achieves fully dynamic visualization that meets the rubric's "Dynamic updates correctly reflect new data" criterion, with robust data persistence and low-latency rendering. Real-time MQTT integration: Asynchronously subscribes to `comp5339/task123/measurements/#`, decoding messages and storing valid data in `session_state` to avoid interface blocking during 0.1s-interval streams. Historical data restoration: On startup, automatically loads `nem_facility_data.csv` (if exists) into `session_state`, validating required columns (e.g., `power_value`, `emission_value`, `facility_code`) to prevent conflicts from old files—ensuring continuity between session

5.2 Core metric and Map rendering

Real-time metric cards consist of four top-aligned cards that calculate and display aggregated values, including Total Power Output, Total CO₂ Emissions, Median Price, and Median Grid Demand, using the latest data from `session_state`, and they feature "Real-time Update" indicators to signal data freshness. The

auto-refreshing map initializes with a default view of Australia (latitude -27.5, longitude 133.8, zoom 4) using OpenStreetMap tiles, with map markers updating dynamically: tooltips adjust their content based on the selected display mode (Power or Emissions), showing the facility name and the corresponding metric (e.g., 85.7 MW or 0.0 tCO₂e), while popups display a comprehensive HTML panel containing static metadata (such as facility code, region, and fuel type) and dynamic metrics (including timestamp, power, emissions, price, and demand), maximizing information density without creating clutter.

5.3 Visual Encoding & Error Handling

Intuitive Marker Design

Map markers use color coding to differentiate fuel types (aligned with sustainability insights):

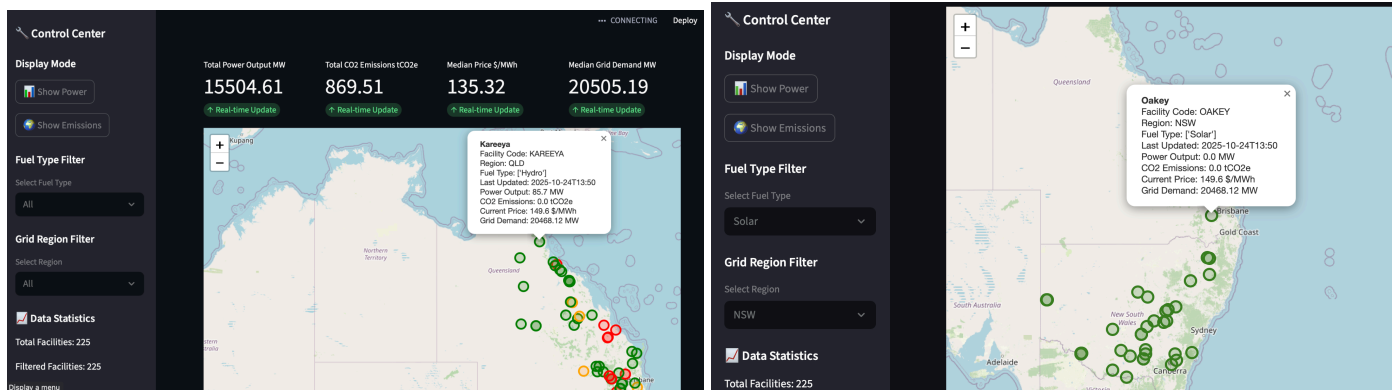
- Green: Renewable fuels (Solar, Wind, Hydro).
- Orange: Gas (transition fuel).
- Red: Coal (high-emission fuel).

Markers use fixed radius (8px) for consistency, with tooltips/popups providing quantitative details—balancing visual simplicity with data richness.

Robust Error Handling

- No matching data: If filters result in zero facilities, the map displays an info message (“No matching facility data...”) to guide users to adjust criteria or wait for MQTT pushes.
- Invalid MQTT messages: Records missing core fields (e.g., `facility_code`, `lat`) are logged and skipped, preventing map glitches or incorrect aggregations.

5.3 Example Visualization



6. Continuous Execution

6.1 Unbounded Loop with Delays

The program runs continuously to simulate an unlimited stream. In each cycle, it retrieves the next batch of records, filtering out new rows using only the (`last_ts`, `last_fac`) cursor, and then emits them according to a strict 0.1-second schedule. After a cycle is completed, we calculate `sleep = max(0, 60s - cycle_runtime)` and block at this time. This ensures that there is at least a 60 second interval between each fetch before the next one is executed. This 60-second delay is added on top of the 0.1-second delay per message, fully meeting the task requirements.

6.2 MQTT Robustness

This loop keeps running without crashing or freezing. The MQTT client uses `QoS=1`, a fixed `CLIENT_ID`, and sets `clean_session=False` for persistence. If the network goes down, any unhandled messages get retried, and there's a Last-Will topic to show the active status. The publisher's timing system is really flexible because delays never cause catch up bursts, it just jumps to the next slot. It keeps the guaranteed time at ≥ 0.1 seconds. In conclusion, the setup is restartable, time-accurate, and reliable to operate, meeting continuous execution standards.

7. Challenges and future improvements

Data Retrieval and Integration: The main challenge was unifying inconsistent schemas across NGER, CER, and ABS datasets. Column normalization and coordinate validation ensured consistency, while missing metadata required minor manual fixes. Future work could automate enrichment via geocoding APIs.

Data Publishing and Continuous Execution: Initial issues with timing, event order, and MQTT reliability were solved using precise scheduling and persistent sessions. Future improvements include crash recovery, monitoring tools, and secure container deployment.

Data Subscription and Visualisation: Key challenges included handling high-frequency MQTT streams (0.1s intervals) without dashboard lag, aligning real-time map rendering with dynamic data, and maintaining filter consistency during rapid refreshes. These were resolved via asynchronous message processing (decoupling ingestion and UI), optimized Folium map re-rendering (updating only changed markers), and anchoring filters in Streamlit's `session_state` to persist user selections.

Appendix1

Team Member Contributions

- Zehao Liu(540876279): Led Data Retrieval (API integration, rate limiting) and Optimization (incremental fetching).
- Dongjie Zhu(550087322): Developed Data Integration and MQTT Publishing (in chronological order with an exact 0.1-second delay for each message), implemented an efficient continuous execution as well.
- Xiaoyan Cao (540866845): Built Dynamic Visualization (Streamlit/Folium dashboard, filters) and Data Subscribing (Assignment 1 integration), earned MQTT bonus points.

Appendix2

Column Name	Data Type	Source Endpoint	Purpose
facility_code	String	/v4/facilities/	Unique identifier for facilities, used as the primary key for integration and data retrieve
facility_name	String	/v4/facilities/	The name for each facility for latter mapping with Assignment1 data
lat/lng	Numeric	/v4/facilities/	Longitude and latitude, Geographic coordinates for spatial visualization
timestamp	ISO 8601	All endpoints	A ISO 8601 time interval within 5 min interval in the current Sydney time zone aka. UTC +11.
Power (MW)	Numeric	/v4/data/facilities/NEM	Instantaneous power output/consumption (MW)
Emission (tonnes)	Numeric	/v4/data/facilities/NEM	CO2 equivalent emissions (tonnes)
Price (\$/MWh)	Numeric	/v4/market/network/NEM	Price per unit of energy (\$/MWh)
Demand (MW)	Numeric	/v4/market/network/NEM	Demand for energy (MW)
state	String	NGER/CER datasets (Assignment 1)	Administrative region of the facility
fuel_list	String	NGER datasets (Assignment 1)	Current and planned fuel technology for using

Appendix3

```
Processing facility: YWNGAHYD
Processing facility: YARWUN
Processing facility: YATSF1
Processing facility: YAWWF
Processing facility: YENDONWF
All data processed!
Total rows after merging: 1038240
Number of negative values in Power (MW) before processing (excluding NaN): 5300
Number of negative values in Emissions (tonnes) before processing (excluding NaN): 0
Number of negative values in Power (MW) after processing (excluding NaN): 0
Number of negative values in Emissions (tonnes) after processing (excluding NaN): 0
/Users/liuzehao/My_university_learning/master/S2-2025Aug/COMP5339_data_engineering/assgn/assignment2/myAttemp/Task1-3.py:426: FutureWarning: DataFrame
eGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded
from the operation. Either pass 'include_groups=False' to exclude the groupings or explicitly select the grouping columns after groupby to silence t
his warning.
  data = data.groupby('facility_code', group_keys=False, observed=True).apply(handle_missing_values_fast)
Total number of facilities before processing: 335
Total number of facilities after processing: 335
Number of missing values in Power (MW) after processing: 0
Number of missing values in Emissions (tonnes) after processing: 0
Number of missing values in Price ($/MWh) after processing: 0
Number of missing values in Demand (MW) after processing: 0
Total rows after cleaning: 675360
/Users/liuzehao/My_university_learning/master/S2-2025Aug/COMP5339_data_engineering/assgn/assignment2/myAttemp/Task1-3.py:524: DeprecationWarning: Cal
lback API version 1 is deprecated, update to latest version
  client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION1, client_id=CLIENT_ID, clean_session=False)
[Main] Waiting for MQTT connection...
[MQTT] connected rc=0
[Main] Connected, starting stream with tick=0.1s
```

Appendix4: Exact 0.1-Second Delay

=== Timing Report ===

Total messages received: 1201

GLOBAL (sender-side, slot_mono_ns): n=1200, mean=0.1000s, min=0.1000, max=0.1000, p50=0.1000, p90=0.1000, p95=0.1000, p99=0.1000, within[0.080,0.120]=100.0%

GLOBAL (sender-side, sent_mono_ns): n=1200, mean=0.1000s, min=0.1000, max=0.1000, p50=0.1000, p90=0.1000, p95=0.1000, p99=0.1000, within[0.080,0.120]=100.0%